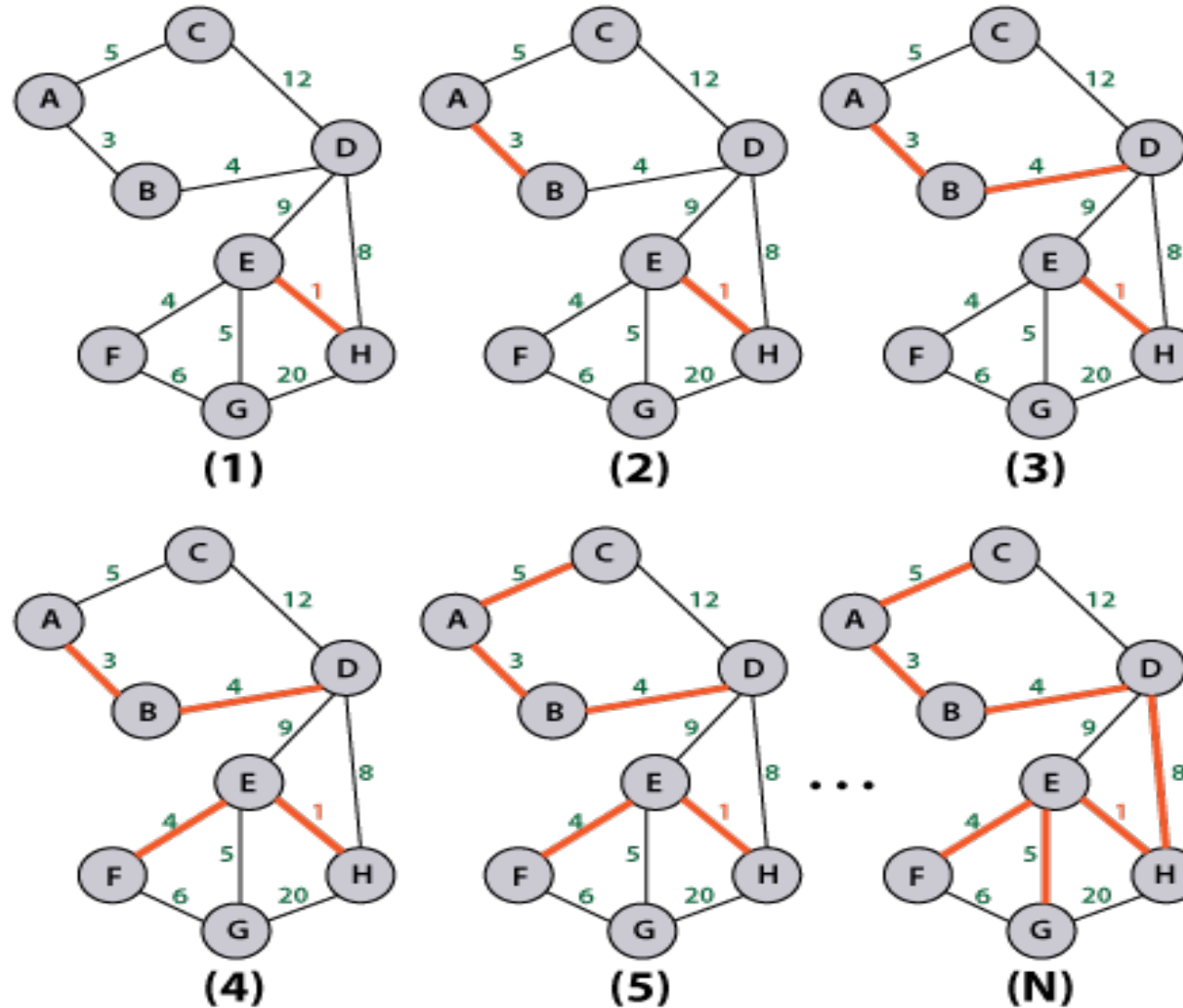




Kruskal's Algorithm

- **Kruskal algorithm** is another most important algorithm used for **Minimum Spanning Tree**.
- MST is a spanning tree having a weight less than or equal to the weight of every spanning tree.
- Kruskal algorithm in Java takes a connected and undirected graph and returns the Minimum Spanning Tree of it.

Kruskal Algorithm



```
import Java.util.*;
```

minimum spanning tree of the given graph

```
class KruskalAlgorithm {
```

```
    class Edge implements Comparable<Edge> {
```

```
        int source, destination, weight;
```

```
        public int compareTo(Edge edgeToCompare) {
```

```
            return this.weight - edgeToCompare.weight;
```

```
        }    };
```

```
    class Subset {
```

```
        int parent, value;
```

```
    };
```

```
    int vertices, edges;
```

```
    Edge edgeArray[];
```

```
    KruskalAlgorithm(int vertices, int edges) {
```

```
        this.vertices = vertices;
```

```
        this.edges = edges;
```

```
        edgeArray = new Edge[this.edges];
```

```
        for (int i = 0; i < edges; ++i)
```

```
            edgeArray[i] = new Edge();
```

```
        }
```

```
        void applyKruskal() {
```

```
            Edge finalResult[] = new Edge[vertices];
```

```
            int newEdge = 0;
```

```
            int index = 0;
```

```
            for (index = 0; index < vertices; ++index)
```

```
                finalResult[index] = new Edge();
```

```
            Arrays.sort(edgeArray);
```

```
            Subset subsetArray[] = new Subset[vertices];
```

```
            for (index = 0; index < vertices; ++index)
```

```
                subsetArray[index] = new Subset();
```

```

for (int vertex = 0; vertex < vertices; ++vertex) {
    subsetArray[vertex].parent = vertex;
    subsetArray[vertex].value = 0;
}
index = 0;

while (newEdge < vertices - 1) {

    Edge nextEdge = new Edge();
    nextEdge = edgeArray[index++];
    int nextSource =
findSetOfElement(subsetArray, nextEdge.source);
    int nextDestination =
findSetOfElement(subsetArray,
nextEdge.destination);

    if (nextSource != nextDestination) {
        finalResult[newEdge++] = nextEdge;
        performUnion(subsetArray, nextSource,
nextDestination);
    }
}

```

```

for (index = 0; index < newEdge; ++index)

    System.out.println(finalResult[index].source + " - " +
finalResult[index].destination + ": " + finalResult[index].weight); }

int findSetOfElement(Subset subsetArray[], int i) {

    if (subsetArray[i].parent != i)

        subsetArray[i].parent = findSetOfElement(subsetArray,
subsetArray[i].parent);

    return subsetArray[i].parent;

}

void performUnion(Subset subsetArray[], int sourceRoot, int destinationRoot)
{

    int nextSourceRoot = findSetOfElement(subsetArray, sourceRoot);

    int nextDestinationRoot = findSetOfElement(subsetArray, destinationRoot);

    if (subsetArray[nextSourceRoot].value <
subsetArray[nextDestinationRoot].value)

        subsetArray[nextSourceRoot].parent = nextDestinationRoot;

```



THANK YOU